A tutorial

The Player

Writing craps strategies in the Craps Sim language

Craps Sim 0.5.1 · grammar version 1

A betting strategy in this simulator does not have to be one of the eleven built in. It can be written: a header, an optional line or two of memory, and an ordered list of rules that say what to do and when. This tutorial covers the whole language — every trigger, every value a rule can read, every action it can ask the table for — building from a two-line strategy to ones that count a shooter's rolls, ride a bet up a ladder, and walk away when a night has given back half its peak.

Every example in these pages was parsed, compiled and simulated by the engine before it was printed. Where an example does something surprising, the surprise is described rather than smoothed over: the traps in a language are worth as much of a tutorial as the features.

Contents

1 Anatomy of a strategy

A strategy is a plain text file. It has three parts, and they must appear in this order:

```
strategy "A name" language 1     <- the header

var hits = 0                     <- declarations: memory, then pressing
press martingale for dont pass

on come-out:                     <- rules, in the order they are considered
    bet pass
```

The header names the strategy and states which grammar it was written against. The version is binding: a file claiming a grammar newer than the engine knows is refused rather than guessed at, so a saved strategy can never quietly change meaning under a later release. Older versions are still read, because every change to the grammar so far has been additive.

Declarations come before the first rule, and the parser will say so if they do not. That ordering is not decoration: it means a rule can never refer to memory that is declared after it.

The smallest strategy that runs

```
strategy "Hello, pass line" language 1

on come-out:
    bet pass
```

Two lines of substance. *On the come-out, bet the pass line.* There is no amount, which means `bet pass` asks for whatever this bet's pressing system currently calls for — under the default flat system, the table minimum. That is how a player says it, and writing an amount you do not care about would be the language describing the engine rather than the game.

Notice what is *not* there. Nothing says to bet again next come-out; the rule is considered at every decision point and does it again. Nothing says to skip the bet when one is already up; `bet` is idempotent, so asking for a bet that is already working is a no-op rather than a second bet. A great deal of a craps strategy is standing behaviour, and standing behaviour is a rule with no bookkeeping around it.

The shape of a rule

```
on <trigger> [when <condition>]:
    <action>
    <action>
```

A rule is a trigger, an optional condition, and one or more actions. The condition belongs to the rule, not to the actions inside it — there is no nested `if`. If you want two conditions, you write two rules. That keeps the text form and the clickable rule editor the same shape, which is the reason both editors can hold the same strategy.

Comments run from `#` to the end of the line. Indentation is decorative — the parser reads a rule body until the next `on` or `for` — but everyone indents, and the renderer will indent for you.

2 Triggers

A trigger says when a rule is considered. A player acts between rolls, so there is one decision point per roll, taken after the previous roll has resolved and before the dice leave the shooter's hand. Every trigger is a question about that moment or about the roll that just finished.

Trigger	Fires when
`session-start`	Once, at the first decision, before any roll
`come-out`	The puck is off — the next roll is a come-out
`point-established`	The roll that just resolved set a point
`point-made`	The shooter just made the point
`seven-out`	The shooter just sevened out and the dice pass
`roll`	Every decision point, unconditionally
`total(n)`	The roll that just resolved totalled n (2 through 12)
`come point on n`	A come flat just travelled to box number n
`dont come point on n`	A don't come flat just travelled to n
`win of <bet>`	That bet just won
`loss of <bet>`	That bet just lost

Three of these repay a second look.

roll fires before the first roll too

The session's opening decision happens before any dice have been thrown, and `on roll` fires there like anywhere else. At that moment `last-total` reads 0 — which is not a total two dice can make.

> A condition written `last-total <= 4` is therefore **true before anything has happened**, and a counter keyed to it starts at one instead of zero. This is a real trap, and it caught a shipped example that counts field numbers. Say what you mean: `last-total >= 2 and last-total <= 4`, since 2 is not a total the dice can fail to reach.

come point on n is a different event from point-established

The table's point and a come bet's point are two different things. A come flat sits in the come box and travels to a box number on the next roll; that travel is `come point on 6`. Without this trigger the only way to notice it was to remember the previous state and compare, which took four rules per number and was easy to get subtly wrong.

win of and loss of follow the bet, not the roll

A pass line bet may take a dozen rolls to resolve. `on win of pass` fires once, when it does. This is what makes progression logic writable without counting rolls yourself. Odds cannot be named here: they resolve with the flat they sit behind and keep no record of their own, so the grammar refuses `on win of odds on pass` rather than quietly treating it as `on win of pass`.

3 Bets and amounts

Thirteen bets can be named, spelled the way craps spells them.

Bet	Notes
`pass / dont pass`	Line bets. Made on the come-out only.
`come / dont come`	Made only with a point on.
`place 4 5 6 8 9 10`	Point on only, and never on the number that is the point.
`hard 4 6 8 10`	Point on only.
`field`	One roll. Renew it every roll.
`any seven / any craps`	One roll.
`odds on pass`	Needs a pass line bet behind it.
`odds on dont pass`	Sized by what it wins, not what it risks.
`odds on come n`	Needs a come point on n.
`odds on dont come n`	Needs a don't come point on n.

Amounts

An amount is optional on `bet` and required on `press` and `regress`. The forms:

Form	Means
`(omitted)`	Whatever this bet's pressing system calls for — same as pressed
`base`	The table's own base stake for this bet
`pressed`	This bet's pressing system, stated explicitly
`max`	The most the odds policy allows. Odds bets only.
`$12  $12.50  $1,200`	Money, to the cent
`2 units / 1 unit / 2 u`	That many table minimums, rounded up to the payout unit
`an expression`	Computed cents, e.g. stake(place 6) * 2

A bare number is cents

This is the one piece of syntax most likely to bite. The engine's unit is the cent, so an unmarked number in an amount position means cents: `bet pass 12` asks for twelve cents, not twelve dollars.

> The table refuses a stake below its minimum rather than quietly rounding it up into one it would take, so this shows in the ledger as `below the table minimum` rather than as the wrong strategy played silently. Still, the habit worth forming is to write `$12` or `2 units` and never a bare number.

`units` and `$` are not the same thing, and the difference matters at the place numbers. `2 units` at a $5 table is $10, rounded up to each bet's payout unit — so the 5 and 9 take $10 and the 6 and 8 take $12, which is the $44 inside a player would actually put up. A flat `$10` on the 6 would be rounded to $12 anyway, but `units` says why.

Odds, and the one asymmetry

```
on roll when point != 0 and up(pass):
    bet odds on pass max
```

`max` collapses the whole 3-4-5x schedule — the per-point lookup and its rounding rules — into one word, because the odds policy belongs to the table and not to the strategy. Note the guard: odds need a flat behind them, so the rule asks whether the pass line is up.

> On the dark side an odds amount means **what it wins**, not what it risks. `bet odds on dont pass $60` lays whatever wins $60 — $120 against the 4 or 10, $90 against the 5 or 9, $72 against the 6 or 8. That is how a player says it at a table, and it is the opposite convention to the right side.

4 Conditions

A condition is any integer expression over what the table can be asked. There is no floating point anywhere in the language: money is cents, everything else is a count, and true is 1.

Operators, loosest binding first

Precedence	Operators
1 (loosest)	or
2	and
3	`<` `<=` `>` `>=` `==` `!=` one per expression; not chainable
4	`+` `-`
5 (tightest)	`*` `/`
unary	not x -x
calls	min(a, b) max(a, b)

Division truncates toward zero. Nothing here can crash: dividing by zero yields zero, and every other operator saturates at the limits of the integer rather than wrapping or trapping. That is deliberate — a strategy is data, and data must not be able to take down a worker thread in the middle of a million-session sweep.

One lexical rule worth knowing

> Binary operators need spaces around them. `dont pass` is two words but `hits-this-shooter` is one, and hyphens are part of names — so `a - b` is subtraction and `a-b` is an identifier that does not exist. A negative literal is the exception: `-200` is a number. This is what lets every bet be spelled the way craps spells it.

What a strategy can read

Thirty-two values, in five groups.

DICE AND POINT	
`point`	The point, or 0 when the puck is off
`come-out`	1 when there is no point
`last-total`	The total of the roll that just resolved (0 before the first)
`roll`	Rolls resolved this session
`rolls-this-shooter`	Rolls since this shooter took the dice
`shooter`	How many shooters have held the dice, counting from 0

MONEY	
`cash`	In hand, in cents
`wealth`	Cash plus the face value of everything on the layout
`profit`	Wealth minus the buy-in
`peak-profit`	The best profit this session has seen

`drawdown`	How far below that peak it now sits
`handle`	Total stakes resolved so far
`buy-in`	What the session started with
`on-table-face`	Face value of everything currently on the felt

THE TABLE

`table-min`	The table minimum, in cents
`table-max`	The table maximum, in cents
`field-12-triple`	1 if the twelve pays 3:1 in the field
`come-odds-work-on-comeout`	1 if come odds stay working on a come-out

THE LAYOUT

`up(bet)`	1 if that bet is on the felt
`stake(bet)`	What is on it, in cents
`working(bet)`	1 if it will resolve during a point cycle
`working-on-come-out(bet)`	1 if it will also resolve on a come-out
`live-come`	Come flats in the box plus come points established
`live-dont-come`	The same on the dark side
`come-point(n)`	The come flat established on n, 0 if none
`dont-come-point(n)`	The don't come flat established on n

HISTORY

`hits(n)`	Times total n has come this session
`hits-this-shooter(n)`	Times n has come since this shooter took the dice
`wins(bet)`	Times that bet has won
`losses(bet)`	Times it has lost
`streak(bet)`	Consecutive wins as a positive run, losses as a negative one
`paid(bet)`	What it was paid on the roll that just resolved

Reads that take a bet or a number are written like calls: `stake(place 6)`, `hits(8)`, `come-point(6)`. The history reads refuse an odds bet, because odds keep no record of their own — `paid(odds on pass)` would have to answer with the line bet's payout, so the grammar refuses the question rather than answering a different one.

Derived history costs nothing when unused. The compiler works out which of these a strategy actually reads and the session maintains exactly those accumulators, so a strategy that never asks about hits pays nothing for the vocabulary.

5 Memory

A strategy may declare counters and flags. They are session-scoped, fixed in number, and reset when a session starts.

```
var hits = 0
var stage = 0
var stake = 500
```

The initial value is honoured — a slot declared `var stake = 500` starts at 500 cents, not at zero. Only a number is allowed after the =; if you want to start from something the table knows, use `on session-start`:

```
var stake = 0

on session-start:
    set stake = table-min
```

Write to a slot with `set`:

```
on win of place 6:
    set hits = hits + 1
```

Three rules about names

A slot may not take a name the grammar already reads as something else — `point`, `cash`, `profit`, `min` and the rest are refused. Otherwise `set roll = 1` would write your slot while every read of `roll` returned the engine's value, and the name would mean two things at once. The parameterized reads are fine as names, though: `hits`, `streak` and `paid` are only reads when a bracket follows, so `var hits` is a perfectly good counter.

Declaring the same name twice is refused, and so is using a name a `for each` block has bound, because a binding is a number the parser substitutes and not a place to store anything.

Thirty-two slots is the ceiling, checked at compile time.

6 How a decision actually works

This is the section to read twice. Every counter, every stage machine and every progression written as rules depends on it, and it is not quite what the shape of the language suggests.

At a decision point the engine walks the rules top to bottom. A rule whose trigger matched and whose condition held contributes its actions. Then the table applies what was collected, in order. Three things follow, and the third is the one that surprises people.

Table reads are a snapshot

`point`, `cash`, `stake(place 6)`, `hits(6)` — every read in the tables above comes from one view built before the first rule is considered, and never rebuilt during the decision. A rule cannot see a bet an earlier rule asked for in the same breath, because nothing has reached the table yet.

Actions are buffered

Every `bet`, `press`, `regress`, `down`, `working` and `leave` goes into a buffer and is applied after every rule has been read, in the order the rules were written. That order decides which bet gets the last dollar when the bankroll is short.

But set is immediate

A `set` writes its slot the moment the statement runs, and every rule below it in the same decision sees the new value. Memory is neither buffered nor part of the snapshot.

So rule order is control flow. Consider:

```
var hits = 0

on win of place 6:
    set hits = hits + 1

on win of place 6 when hits >= 3:
    down place 6
```

The second rule sees the count the first one just wrote, so the bet comes down on the **third** hit. Swap the two rules and it comes down on the **fourth**, because the guard would read the count as it stood before this win. Both orders parse, both compile, both run, and the Bench shows both rules firing either way. The difference is only in the money.

The practical technique

When several rules update one counter, write them highest-state-first so a single decision cannot cascade through all of them:

```
on roll when count == 4 and was-point-number == 1:
    set count = 5

on roll when count >= 1 and count <= 3:
    set count = count + 1

on roll when count == 0 and was-point-number == 1:
    set count = 1
```

Written the other way round, a roll would take the count from 0 to 1, then 1 to 2, then 2 to 3 — three steps for one roll. This is the 5-Count in section 13, and it is the single most useful idiom in the language.

The other half of the technique is a one-shot flag. A rule fires at every decision where its condition holds, so `on roll when hits >= 2: regress ...` will ask again on every subsequent roll and be refused each time. Guard it with a stage variable and set the stage in the same rule:

```
on roll when hits >= 2 and stage == 0:
    regress place inside to base
    set stage = 1
```

7 Pressing

There are two ways a bet's stake changes, they happen at different moments, and knowing which is which is most of understanding this language.

A pressing system, declared per bet stream

Declared once, before the rules, and applied where the bet resolves — the moment the dealer pays you and asks whether to press, before the next roll:

```
press martingale                 # every stream
press half-press for hard 6      # just this one
press martingale for dont pass
```

Twelve systems are available: `flat`, `full-press`, `half-press`, `press-and-pull`, `paroli-3`, `1-3-2-6`, `martingale`, `grand-martingale`, `dalembert`, `reverse-dalembert`, `fibonacci` and `oscars-grind`. The default is flat.

Being able to say *Martingale the don't pass and leave the place bets flat* is the point of per-stream declarations; before them the progression was one global choice.

A rule, at the decision point

```
press place 6 to $18            # a destination
press place 6 by 1 unit         # a step from wherever it stands
regress place inside to base
```

`press` only raises and `regress` only lowers. Asking a press to lower a bet is refused with a reason rather than silently ignored — the verb should mean what it says. `by` is sugar: it becomes the destination it names, computed from what is on the bet.

How the two interact

A flat stream does not re-price a winner — flat means nobody is pressing this bet, not that the table drags it back to base every time it wins. So under the default, a stake a rule sets stays where the rule put it, and a ladder climbs:

```
on win of place 6 when stake(place 6) < $24:
    press place 6 by 1 unit
```

Under a real pressing system the system owns the stake: it re-prices the bet where the bet resolves, and a rule may then override at the next decision point. Last write wins. **If you are pressing from rules, leave that stream flat** and let the rules do the work.

8 Taking bets down, off, and leaving

```
down place 6            # take it down; the stake comes back
working place 6 off     # leave it there, resolving nothing
working place 6 on
working place 6 on come-out
leave "a reason for the reader"
```

`down` returns the stake to the rail. It is refused for a contract bet — once the point is on, the pass line and a come flat are committed, and no table gives them back. Odds always come down; that is most of why they are the best bet on the table.

`working` is the other thing entirely: the bet stays on the felt, still your money, still counted in what you would walk away with, but resolving nothing. Only place bets and hardways can be called off.

Working on the come-out

Craps has always had two working questions and they have opposite defaults. A bet works during a point cycle unless you call it off. A bet does *not* work on the come-out unless you call it on. The qualifier is how you say the second:

```
working place 6 on              # during a point cycle (the default)
working place 6 on come-out     # on the come-out as well
```

A bet that is off is off everywhere — the two flags are combined, not independent. And the trade is exactly what it looks like: the seven that wins the line takes your working place bets with it.

Leaving

`leave` ends the session on the strategy's own terms. The player walks with cash plus the face value of whatever is still on the layout, exactly as a session reaching its horizon does. The quoted reason is for whoever reads the strategy; the engine discards it.

> A stop rule fires at the next decision point, which is **after** the roll that crossed the threshold — so a session can overshoot by one roll's swing. That is not a defect, it is what happens to a live player too, but it is worth knowing when a stop-loss at $200 walks out at $211.

9 Groups and blocks

Two ways to avoid writing the same rule six times.

Groups: one phrase, several bets

Group	Members
`place inside`	5, 6, 8, 9
`place outside`	4, 10
`all place`	4, 5, 6, 8, 9, 10
`all hardways`	hard 4, 6, 8, 10
`everything`	all six place numbers and all four hardways

`everything` is what a dealer would sweep when you are leaving: the bets that can actually come down. Not the line, which is a contract once the point is on, and not the one-roll propositions, which resolve before the question could be asked.

```
bet place inside 2 units
down everything
working all hardways off
```

A group is sugar, expanded at parse time into one statement per member. It carries one shared amount and one shared condition, which is its limitation: you cannot say *place the inside numbers except the one that is the point* with a group, so the rule above is refused for whichever number is the point. A refused bet costs nothing and the ledger names the reason, but if you would rather not see it, write a block instead.

Blocks: a rule written once, produced per number

```
for each of 4, 5, 6, 8, 9, 10 as n {
    on roll when point != 0 and point != n:
        bet place n
}
```

The block is read once per value with `n` bound to it, so what comes out is exactly the rules you would otherwise have typed six times. The binding may stand anywhere a number may: `place n`, `hard n`, `odds on come n`, `come-point(n)`, `total(n)`, or as a bare term in an expression. Blocks nest, and an inner binding shadows an outer one.

Walking two lists in step

A pair of numbers often has something to say about each other. A second list, walked alongside the first, says it once:

```
for each of 6, 8 as n with 8, 6 as other {
    on win of place n when hits-this-shooter(n) >= 2:
        down place other
}
```

The 6 winning takes down the 8, and the 8 takes down the 6. Lists walked together must be the same length, and a name may not be bound twice.

A block is presentation, not meaning. The tree holds the expanded rules; the block remembers that a person wrote them once, keeping its own text verbatim including comments. Whether it is still a block is never recorded — it is asked, by re-reading that text and comparing. So unfolding a block to look at it changes nothing, and editing two of its iterations apart stops it being one block at the moment they differ.

10 The table adjudicates

A strategy never moves money. It proposes; the table validates every proposal against point state, the odds policy, payout units, the table maximum and the bankroll, then applies it or refuses it with a reason. There is exactly one place where money moves onto the layout, and it is not in the language.

What the table will not do, and when

Rule of craps	The refusal reads
Line bets are made on the come-out only	the point is already established
Come, don't come and hardways need a point	there's no point yet
Place bets need a point	there's no point yet
No place bet on the number that is the point	that number is the point
Odds need a flat behind them	not allowed right now
Contract bets can't come down with a point on	a contract bet can't come down
Only place bets and hardways can be called off	not allowed right now
A stake below this bet's minimum	below the table minimum for that bet
A press that lowers, or a regress that raises	a press can't lower
Nothing there to press, take down or turn off	there's nothing on that bet
The odds policy allows none behind this point	odds policy allows none
Neither stake nor base fits the bankroll	bankroll won't cover it

Every one of these is an event, not a silence. The Replay ledger shows the verb that was attempted, the bet, the amount asked for and the reason — because a strategy that quietly does nothing and returns a confident distribution is the worst thing this surface could produce.

A stake above the table maximum is not refused; it is clipped to the maximum and the clip is announced. That is how a real table stops a Martingale, and it is shown rather than left to be inferred from a flat spot in a curve.

Refusals are not necessarily bugs

Some are the natural cost of a concise rule. `bet place inside` will be refused for whichever number is the point, every point cycle, and that is fine — a refused bet moves no money. The question to ask of a refusal is whether it means what you intended: a place bet refused with *there's no point yet* on every come-out is expected; a press refused with *there's nothing on that bet* usually means a bet you thought was up is not.

11 Writing for a table you have not seen

A strategy written in dollars is true at one table and quietly wrong at every other. Four reads let a strategy be written in terms of the table it is actually played at.

```
on roll when profit <= 0 - buy-in / 2:
    leave "half the buy-in gone"

on loss of pass when stake > table-max:
    set stake = table-min

on come-out when field-12-triple == 0:
    leave "this table pays the twelve double"
```

`buy-in` makes a stop-loss a fraction rather than a figure. `table-max` lets a progression see its own ceiling coming and start over, instead of pushing into a bet the table will truncate. And `field-12-triple` and `come-odds-work-on-comeout` let a strategy whose arithmetic assumes a particular layout decline a table that does not have it, rather than playing on and reporting the numbers as though it had the one it meant.

> There is no clock. The language has no wall time by design — evaluation is a pure function of the table, the events since the last decision, and memory, which is what every paired comparison in the app depends on. If you want *about two hours*, count rolls: `on roll when roll >= 220: leave` at a typical pace. It is an approximation, and worth labelling as one.

12 Debugging: the Bench and the static checks

The compiler answers some questions before a die is thrown, and refuses only one thing.

Refused: a strategy that never bets

This strategy never places a bet, so there is nothing to simulate. That is a compile error, because a strategy that cannot put money at risk has nothing to say. Note that a `press` does not count — pressing an empty slot is always refused, so a strategy whose only action is a press can never risk a cent.

Reported, but never blocking

Everything else the compiler notices is a sentence in the order-ticket strip beside the editor, not a refusal:

Check	Says
Dead rule	A condition that can never hold — asking one number to be two things, or a window with nothing in it
Conflict	Two unconditional rules acting on the same bet at the same trigger, and which one wins
Exposure	The most this strategy can have on the layout at once, against the budget
Clipping	The step at which a pressing system meets the table maximum
Cost	Instructions per decision in the worst case

None of these stops a run, and that is deliberate. A dead rule is legal. A hedge that cannot hedge is legal. The app exists to model a belief exactly and draw what happens, and a compiler that refused unsound play could not refute unsound play. The dead-rule check is also not a theorem prover, and says so: it folds the arithmetic people actually write and looks for the two contradictions people actually make.

The Bench

The real debugger is on the Replay screen: one session on a fixed seed, stepped roll by roll, with every rule that fired marked, every cent attributed to the rule that asked for it or to the table, every refusal listed with a way to reach it, and a fire count per rule for the whole run.

> **The fire count is the first thing to look at.** A rule showing `0×` never fired, and that is nearly always the bug: a condition that cannot hold, a trigger that does not mean what you thought, a counter that never reached its threshold. Every defect found in the shipped examples of this language was found this way, and none of them produced an error message — they all compiled, ran, and returned plausible-looking money.

The second thing to look at is the refusals, and the third is whether the counts are in the ratios dice make. A rule on the 6 and 8 firing far more often than one on the 4 and 10 is right; the reverse means something is wrong upstream.

13 Worked examples

Twelve strategies, every one parsed, compiled and simulated before printing. They are demonstrations of the language, not advice — most of them are bad bets and one is deliberately superstitious.

Pass line BASIC

The smallest strategy that plays. One standing rule; no bookkeeping.

```
strategy "Hello, pass line" language 1

on come-out:
    bet pass
```

Pass line with full odds BASIC

The guard is doing real work: odds need a flat behind them, so the rule asks whether the pass line is up. `max` takes the whole 3-4-5x schedule from the table.

```
strategy "Pass with full odds" language 1

on come-out:
    bet pass

on roll when point != 0 and up(pass):
    bet odds on pass max
```

3-Point Molly BASIC

Two come bets working alongside the line, each backed with odds as it travels. Nine rules and no memory at all — `live-come` caps the come bets, and idempotent `bet` makes replacement automatic when one repeats.

```
strategy "3-Point Molly" language 1

on come-out:
    bet pass

on roll when point != 0 and up(pass):
    bet odds on pass max

on roll when point != 0 and live-come < 2:
    bet come

for each of 4, 5, 6, 8, 9, 10 as n {
    on come point on n:
        bet odds on come n max
}
```

Iron Cross BASIC

Every number except the seven pays something. Note the field renewing on every roll with no special vocabulary — a one-roll bet under a standing rule simply is a bet made every roll — and the point-exclusion guard on each place number.

```
strategy "Iron Cross" language 1

on roll when point != 0 and point != 5:
    bet place 5

on roll when point != 0 and point != 6:
    bet place 6

on roll when point != 0 and point != 8:
    bet place 8

on roll when point != 0:
    bet field

on roll when profit >= $150 or profit <= -$150:
    down everything
    leave "enough"
```

$44 inside, regressed INTERMEDIATE

A group hit counter: any of four numbers advances the same count. The stage flag makes the regression fire once rather than at every subsequent decision. This version accepts the refusals that come from placing the group including the point.

```
strategy "44 Inside, regressed" language 1

var hits = 0
var stage = 0

on point-established:
    set hits = 0
    set stage = 0
    bet place inside 2 units

for each of 5, 6, 8, 9 as n {
    on win of place n:
        set hits = hits + 1
}

on roll when hits >= 2 and stage == 0:
    regress place inside to base
    set stage = 1

on roll when hits >= 4 and stage == 1:
    down all place
    set stage = 2

on seven-out:
    set hits = 0
    set stage = 0
```

The same, without the refusals INTERMEDIATE

A block gives each number its own `point != n` guard, which a group cannot carry. Ten rules instead of eight, and a clean ledger.

```
strategy "44 Inside, without the refusals" language 1

var hits = 0
var stage = 0

for each of 5, 6, 8, 9 as n {
    on roll when point != 0 and point != n:
        bet place n 2 units

    on win of place n:
        set hits = hits + 1
}

on roll when hits >= 2 and stage == 0:
    regress place inside to base
    set stage = 1

on seven-out:
    set hits = 0
    set stage = 0
```

A Martingale that knows the ceiling INTERMEDIATE

Memory as a bet size, and the reset leg reading the table's own maximum. The ordering is load-bearing: the doubling rule runs first, so the guard below it sees the already-doubled stake and asks whether *that* now exceeds the maximum. Run it and the ceiling rule shows **0x** — and this is the case where that is not a bug. Climbing from $5 past a $5,000 ceiling takes ten straight losses and costs $5,115 on the way; the stop-loss at half a $1,000 buy-in ends the session first, every time. The rule guards a state the bankroll cannot reach, which is worth knowing before trusting a ceiling to save you.

```
strategy "Martingale that knows the ceiling" language 1

var stake = 0

on session-start:
    set stake = table-min

on come-out:
    bet pass stake

on loss of pass:
    set stake = stake * 2

on loss of pass when stake > table-max:
    set stake = table-min

on win of pass:
    set stake = table-min

on roll when profit <= 0 - buy-in / 2:
    leave "half the buy-in gone"
```

The 5-Count ADVANCED

The Captain's shooter-qualification rule, and the clearest use of the highest-state-first idiom: written the other way round, one roll would cascade through every counter rule. Since a condition is one expression and cannot be broken across lines, the point-number test is computed into memory first — a block writes the flag, and because `set` is immediate, every rule below reads it.

```
strategy "The 5-Count" language 1

var count = 0
var was-point-number = 0

on seven-out:
    set count = 0

# Recompute, at every decision, whether the roll that just resolved was
# a point number. `set` takes effect the moment it runs, so every rule
# below these sees the answer -- memory standing in for the condition
# the language has no other way to phrase in one line.
on roll:
    set was-point-number = 0

for each of 4, 5, 6, 8, 9, 10 as n {
    on roll when last-total == n:
        set was-point-number = 1
}

# Highest state first. Written the other way round, a single roll would
# take the count 0 -> 1 -> 2 -> 3 in one decision.
on roll when count == 4 and was-point-number == 1:
    set count = 5

on roll when count >= 1 and count <= 3:
    set count = count + 1

on roll when count == 0 and was-point-number == 1:
    set count = 1

on come-out when count >= 5:
    bet pass

on roll when count >= 5 and point != 0 and up(pass):
    bet odds on pass max
```

Press and ride the 6 and 8 ADVANCED

A bet whose size depends on its own history, climbing by one unit per hit to a $24 cap. `by` saves computing the destination by hand, and the `stake(place n) < $24` guard is what stops the ladder.

```
strategy "Press and ride the 6 and 8" language 1

for each of 6, 8 as n {
    on roll when point != 0 and point != n:
        bet place n

    on win of place n when stake(place n) < $24:
        press place n by 1 unit
}

on roll when profit >= $100:
    down everything
    leave "took the ride"
```

Don't pass, laying the odds ADVANCED

The dark side, with the win-target odds convention: `$60` lays whatever wins $60 at each point. The exit counts points made against the player across shooters, and the counter rule sits below the leave rule deliberately — so the leave sees the count as it was, and fires on the second.

```
strategy "Don't pass, laying the odds" language 1

var made = 0

on come-out:
    bet dont pass

on roll when point != 0 and up(dont pass):
    bet odds on dont pass $60

on point-made when made >= 1:
    leave "two straight points against me"

on point-made:
    set made = made + 1

on seven-out:
    set made = 0
```

Working through the come-out ADVANCED

Paired bindings and both working flags. Each number is called on for the come-out only once, guarded by reading the flag back, and each number takes its partner down after two hits.

```
strategy "Working through the come-out" language 1

for each of 6, 8 as n with 8, 6 as other {
    on roll when point != 0 and point != n:
        bet place n

    on roll when up(place n) and working-on-come-out(place n) == 0:
        working place n on come-out

    on win of place n when hits-this-shooter(n) >= 2:
        down place other
}
```

Hardways with a craps hedge ADVANCED

Per-stream pressing on two bets, group working toggles across the come-out, a `total(7)` trigger, and a losing streak read. Turning the hardways off for the come-out is what a player does with them.

```
strategy "Hardways with a craps hedge" language 1

var sevens = 0

press half-press for hard 6
press half-press for hard 8

on come-out:
    bet any craps
    working all hardways off

on point-established:
    bet hard 6
    bet hard 8
    working all hardways on

on total(7) when point == 0:
    set sevens = sevens + 1

on roll when streak(hard 6) <= -3:
    down hard 6

on roll when sevens >= 3:
    leave "three come-out sevens is enough excitement"
```

Bankroll discipline ADVANCED

Four ways to stop, none of them a dollar figure: a trailing stop off `peak-profit`, a stop-loss as a fraction of the buy-in bounded by `min`, a fatigue rule on rolls and shooters, and a churn limit on `handle` — leave once total stakes resolved reach half again the buy-in, however the night is going.

```
strategy "Bankroll discipline" language 1

on come-out:
    bet pass

on roll when point != 0 and up(pass):
    bet odds on pass max

on roll when peak-profit >= buy-in / 4 and profit * 2 <= peak-profit:
    leave "gave back half the peak"

on roll when profit <= 0 - min(buy-in / 2, $300):
    leave "stop loss"

on roll when rolls-this-shooter >= 40 and shooter >= 3:
    leave "long night"

on roll when handle >= buy-in * 3 / 2:
    leave "churned half again the buy-in"
```

A Complete reference

Grammar

```
strategy "<name>" language 1

var <name> [= <number>]                     zero or more, before any rule
press <system> [for <bet>]                  zero or more, before any rule

on <trigger> [when <expr>]:                 one or more rules
    <action>...

for each of <n>, <n>... as <name>           blocks may nest
     [with <n>, <n>... as <name>] {
    <rules>
}
```

Actions

Action	Meaning
`bet <bet> [<amount>]`	Put it up if it is not up; top odds toward a target
`press <bet> to <amount>`	Raise to a stake
`press <bet> by <amount>`	Raise by a step from where it stands
`regress <bet> to <amount>`	Lower to a stake
`down <bet>`	Take it down; the stake returns
`working <bet> on|off`	Resolve, or sit there, during a point cycle
`working <bet> on|off come-out`	The same question for the come-out roll
`set <var> = <expr>`	Write memory; takes effect immediately
`leave ["reason"]`	End the session on the strategy's own terms

Pressing systems

```
flat               full-press          half-press
press-and-pull     paroli-3            1-3-2-6
martingale         grand-martingale    dalembert
reverse-dalembert  fibonacci           oscars-grind
```

Limits, all checked before a die is thrown

Limit	Value	Why
Memory slots	32	Per-session state is a fixed array
Actions per decision	48	The proposal buffer is fixed size
Rules per strategy	512	Every rule is walked at every decision
Expression nesting	64	The parser walks it on the native stack
Block nesting	8	Blocks multiply rules by their value lists

Limit	Value	Why
Operand stack depth	32	The compiled machine's fixed depth

What the language deliberately cannot do

— No recursion, no unbounded loops, no dynamic allocation, no I/O. Every strategy is total, terminating and cost-bounded before it runs — which is what makes it safe to execute an unreviewed one 1.2 million times across every core.

— No clock, no random numbers, no ambient state. Evaluation is a pure function of the table, the events since the last decision, and memory. Every paired comparison in the app depends on it.

— No mid-roll or call bets. The decision point is between rolls, because that is where a player's decisions are.

— No buy, lay, place-to-lose, hop or horn bets yet. The grammar is designed so each is one enum variant plus its resolution; adding them is its own piece of work.

— Placing the number that is the point is refused unless the table is configured to allow it. A real table will usually sell it; this one asks first.

Further reading

The design specification is `docs/STRATEGY_DSL.md` in the repository, which states the principles this language is built on and records, beside each one, where building it proved the original claim wrong. `docs/STRATEGY_DSL_REVIEW.md` is a review of the implementation with its own account of what was fixed and what was deliberately left alone. The worked examples in section 13 live in `crates/craps-engine/src/strategy/examples.rs`, where the test suite compiles and simulates each one.